



操作系統安全

李琦

清華大學網研院



本章的內容組織



第一節 基礎攻擊方案

- 內存管理基礎
- 內存棧區攻擊方案
- 內存堆區攻擊方案

內存為對象的攻擊
以內存為對象的簡單
攻擊方案



第二節 基礎防禦方案

- W^X (NX, DEP)
- ASLR
- Stack Canary
- SMAP, SMEP

內存粒度的防禦方案
防禦基礎內存攻擊的
方案



第三節 高級控制流劫持技術

- 進程執行細節
- 面向返回地址編程
- 全域偏置表劫持
- 虛假vtable劫持

惡意修改進程行為
基于第一章的攻擊
繞過第二章基礎防禦



第四節 高級系統保護方案

- 控制流完整性
- 指針完整性
- 信息流控制
- I/O子系統保護

系統粒度的防禦方案
通過更嚴格的安全機
制防禦控制流劫持

有效防禦

成功繞過

有效防禦



第1節 操作系統基礎攻擊方案

- ✓ 1.1 操作系統內存管理基礎
- ✓ 1.2 基礎的棧區攻擊方案
- ✓ **1.3 基礎的堆區攻擊方案**

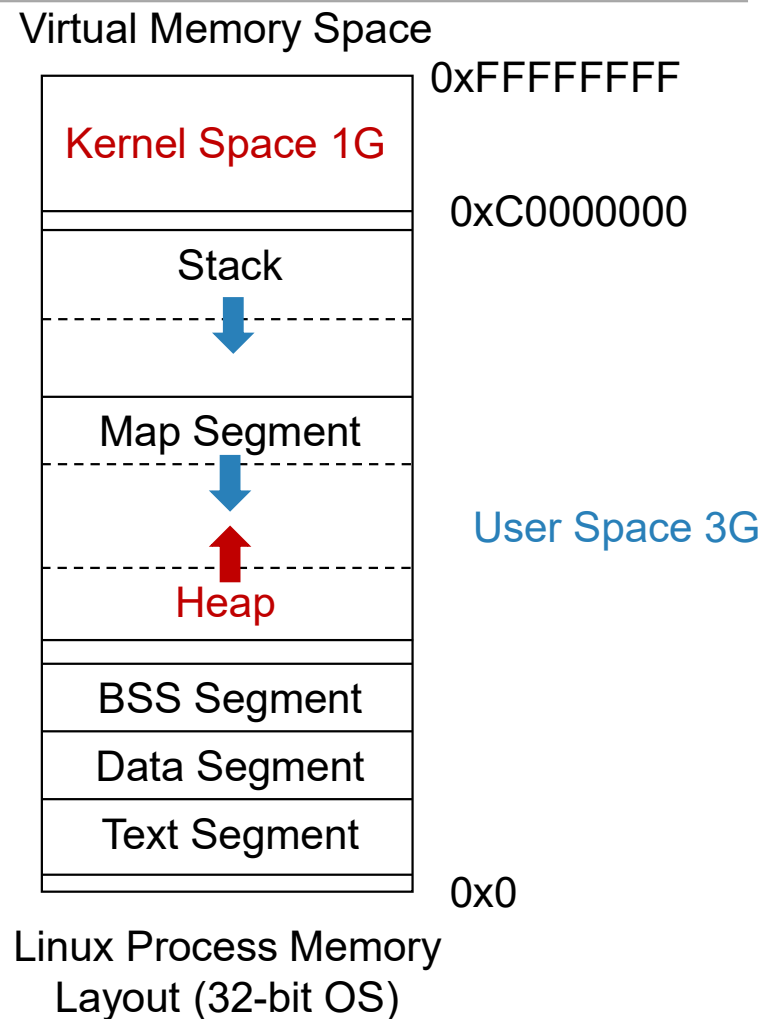


正常工作的堆管理器

在程序運行過程中，堆可以提供動態分配的內存，允許程序申請指定大小的內存

堆區是程序虛擬地址空間的一塊連續的綫性區域，它由低地址向高地址方向增長

我們一般稱管理內存堆區的程序為**堆管理器**
稱堆管理器分配的最小內存單元為**堆塊 (Chunk)**





正常工作的堆管理器

- **堆管理器**處于用戶程序與內核中間地位，主要做以下工作：
 1. **響應用戶的申請內存請求**。向操作系統申請內存，然後將其返回給用戶程序；堆管理器會預先向內核申請一大塊連續內存，然後過**堆管理算法**管理這塊內存；當出現了堆空間不足的情況，堆管理器會再次與內核行交互
 2. **管理用戶所釋放的內存**。一般情況下，用戶釋放的內存并不是直接返還給操作系統的，而是由堆管理器進行管理；這些釋放的內存可以來響應用戶新申請的內存的請求

堆管理器的緩衝作用顯著降低了動態內存管理的性能開銷



正常工作的堆管理器

- 堆管理器通常不屬操作系統內核的一部分，而是屬標準C函數庫的一部分，根據標準C函數庫的實現而采用不同堆管理器

ptmalloc2多綫程堆管理器，是glibc的堆管理器，是最被廣泛使用的堆管理器，應用于絕大多數的Linux發行版上

- 其他常見的堆管理器有：
 - dlmalloc堆管理器為glibc的早期堆管理器，所有綫程共享同一堆區
 - musl堆管理器，適配于嵌入式系統

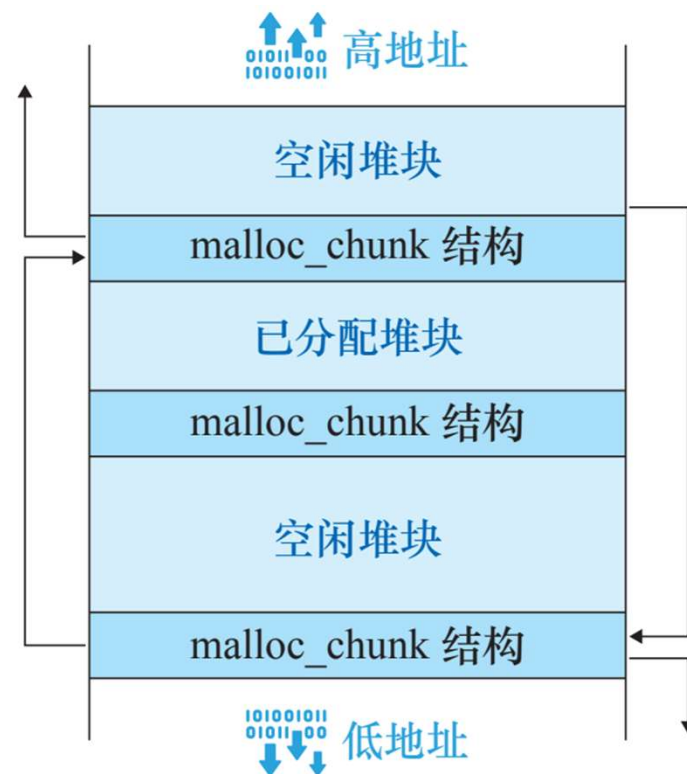
這些堆管理器的根本區別在于堆管理算法和管理元數據



正常工作的堆管理器

- ptmalloc2 管理堆區內存的最小單元是堆塊 (Chunk) 是向內核申請和歸還的最小單元
- 每一個Chunk分為頭部數據結構為malloc_chunk結構體 (低地址)，之後為分配或者未分配的數據塊 (高地址)
- 空閑堆塊 (Free Chunk) 之間通過雙向鏈錶鏈接，并根據大小由5個Bin分類組織

堆管理算法直接操縱malloc_chunk結構和5個Bin實現堆區的內存管理





正常工作的堆管理器

• 堆管理元數據結構 malloc_chunk:

1. prev_size: 當上一個Chunk為空閑，存儲上一個Chunk大小，否則存上一個Chunk的數據
2. size: 該Chunk大小
3. NON_MAIN_ARENA: 是否屬子線程
4. IS_MMAPPED: 是否由mmap分配
5. PREV_INUSE: 前一個Chunk是否被分配
6. bk, fd: 鏈接Bin當中空閑塊的前後向鏈表指針，只有在空閑時使用

malloc_chunk 结构



面向堆區攻擊方案的核心在于: 如何惡意操縱堆管理數據結構



堆區溢出攻擊

堆溢出攻擊

是一類攻擊者越界訪問并篡改堆管理數據結構，實現惡意內存讀寫的攻擊

- 堆區溢出攻擊是堆區**最常見**的攻擊方式，這種攻擊方式可以實現惡意數據的覆蓋寫入，進而實現進程控制流劫持

堆溢出的使用廣泛，25%針對windows7的攻擊都是堆區溢出

Heap Vulnerability	Occurrences
Heap Overflow	673
Use-After-Free	264
Heap Over-Read	125
Double-Free	35
Invalid-Free	33

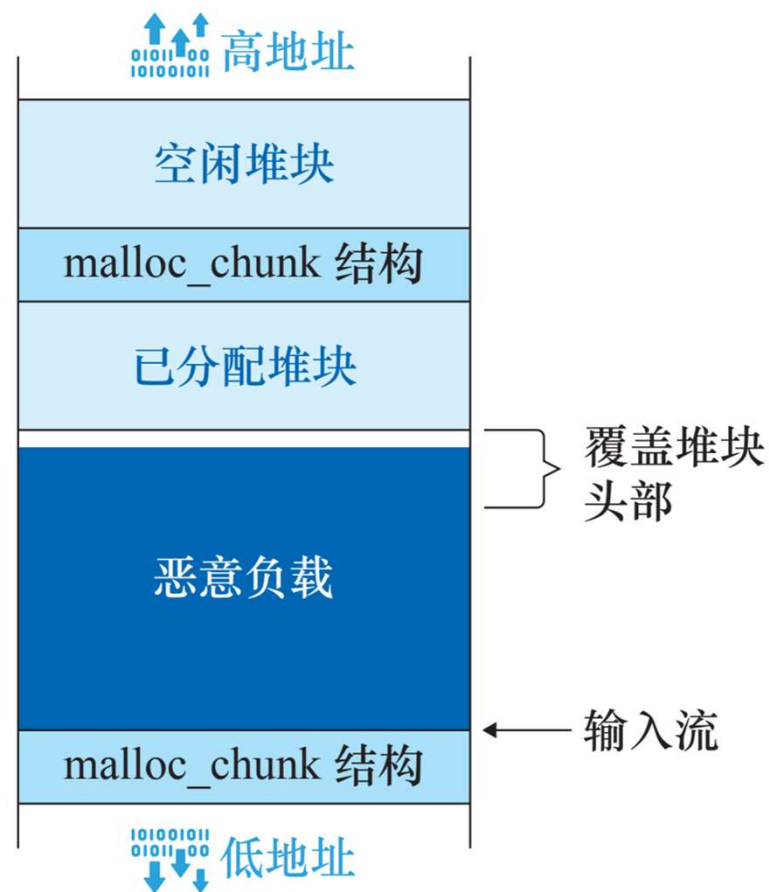
注. 數據來自CVE-2017當中的檢索結果



堆區溢出攻擊

- 最簡單的堆區溢出：

直接覆蓋malloc_chunk首部為無意義內容，在堆管理器處理管理元數據時將造成崩潰



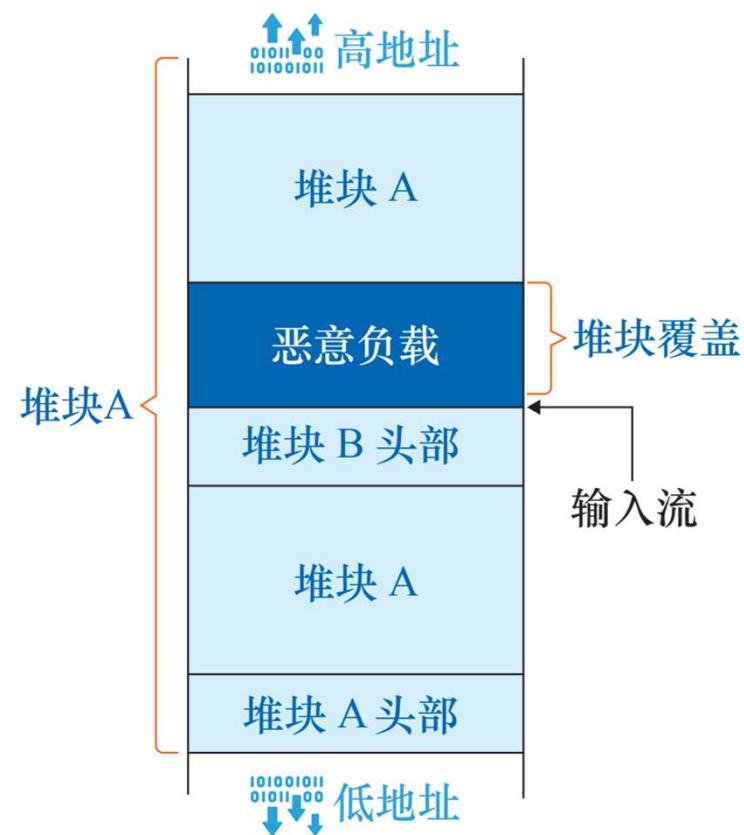


堆區溢出攻擊

- 堆區溢出：構造堆塊重疊 (Heap Overlap)

堆塊重疊是一種**病態**堆區內存分配狀態，
同一堆區邏輯地址被堆管理器**多次分配**

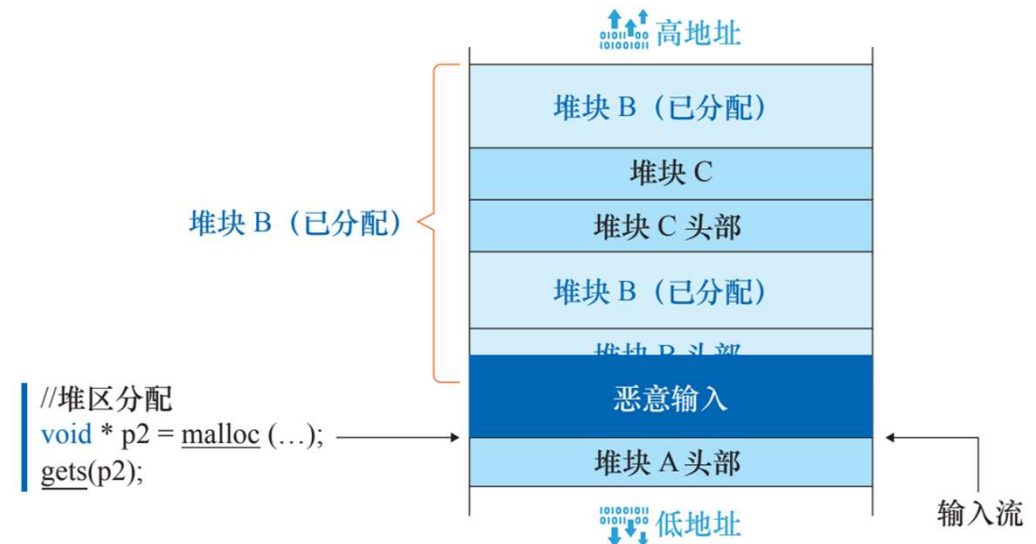
如右圖所示，造成Heap Overlap之後攻擊者可以通過寫入一個堆塊，實現對另一堆塊內容的寫入；同理，讀出被覆蓋堆塊當中的數據





堆區溢出攻擊

- 我們考慮一個案例：堆塊A是可以發生溢出的堆塊，其中B和C是被分配（allocated）狀態的塊，而且C是我們的攻擊目標塊
- 攻擊者首先通過堆區溢出數據去改寫Chunk B的size域，把Chunk C包含到Chunk B當中，以此構造了堆塊重疊

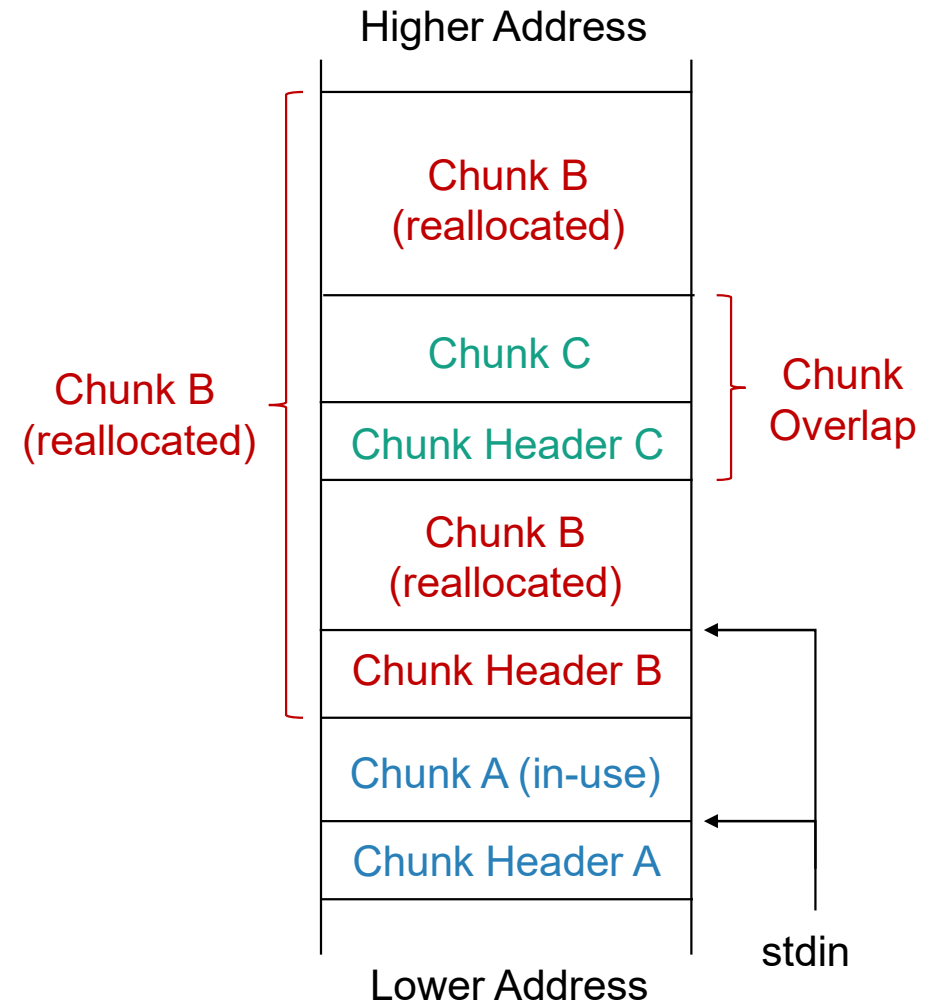


注. size字段在malloc_chunk結構的第一個字節（管理已分配堆塊時），因而僅溢出1字節就可以覆蓋到size域



堆區溢出攻擊

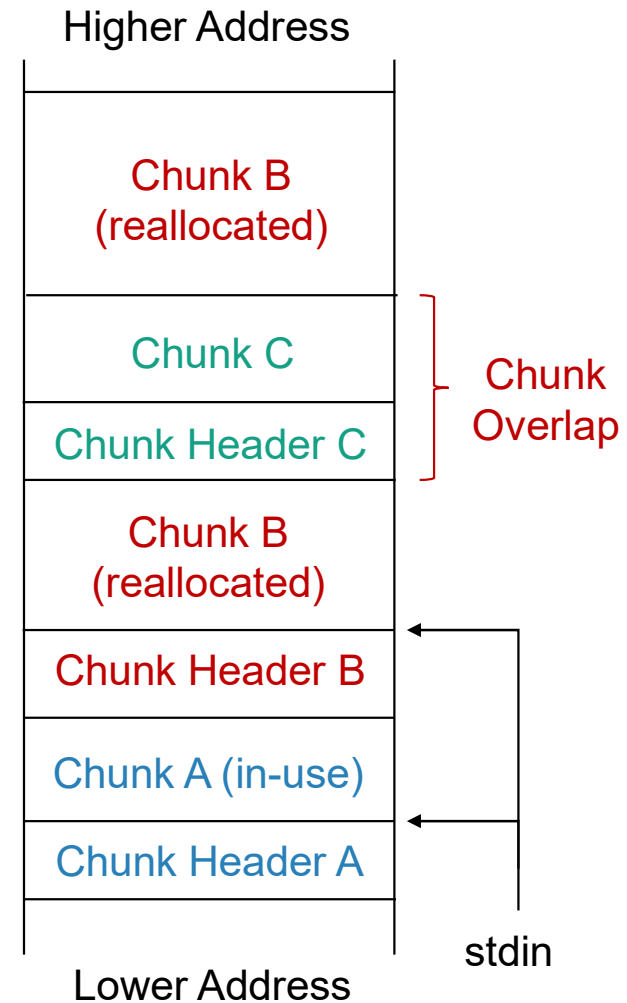
- 而後攻擊者操縱控制邏輯，使被修改了size字段的Chunk B將被重新分配
- 最終構造堆塊重疊，攻擊者可以通過讀/寫被重新分配的Chunk B來讀/寫塊Chunk C當中的數據





堆區溢出攻擊

- 總結，上述構造堆塊覆蓋的方案需要受害程序滿足以下的條件：
 - ① 存在錯誤的輸入檢查邏輯，使攻擊者可以進行溢出（溢出1個字節即可）
 - ② 對應的受害程序應能產生符合條件的堆區布局也就是連續分配的三個堆
 - ③ 且攻擊者可以操縱控制流實現Chunk B的釋放和重新分配，並能在分配後進行讀寫





堆區溢出攻擊

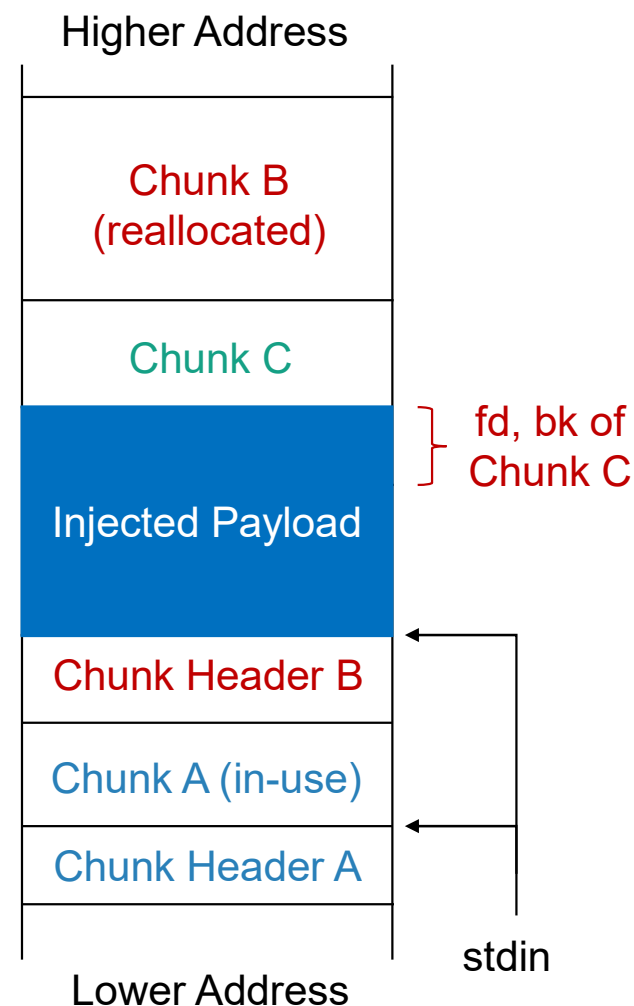
- 更加複雜的堆區溢出攻擊：利用堆管理其他機制

例如，基于unlink機制的堆區溢出攻擊：

unlink宏從空閑堆塊構成的雙向鏈表中，提取空閑堆塊，而後返回給用戶空閑堆塊

攻擊者將設法用溢出數據覆蓋前/後向鏈表指針（fd，bk 字段），在unlink宏調用時可以將任意內存地址當作未分配的堆塊使用；

最終，攻擊者可讀/寫任意內存區域（write-everything-anywhere），并以此為基礎進行更複雜的攻擊



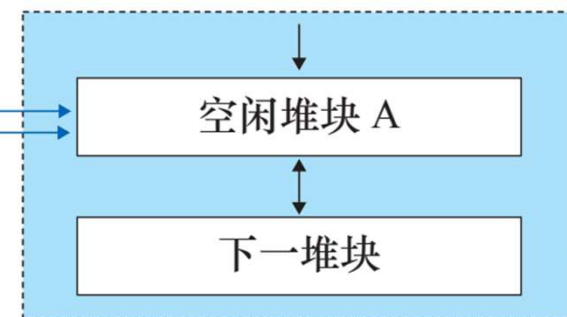


堆區的其他攻擊：Use-After-Free

- **Use-After-Free** 是進程由于實現上的錯誤，**使用已被釋放的堆區內存**，UAF是一種存在廣泛的漏洞，僅2020年上半年，**CVE當中就彙報了超過90種UAF漏洞**

被free函數釋放的堆塊內存仍然可以被繼續使用，當再次調用malloc分配內存時，會同時有兩個指針指向同一堆塊造成 **堆塊重疊**

```
// 空懸指針  
free(p1);  
...  
void * p2 = malloc(...);  
// 已分配堆塊 A
```



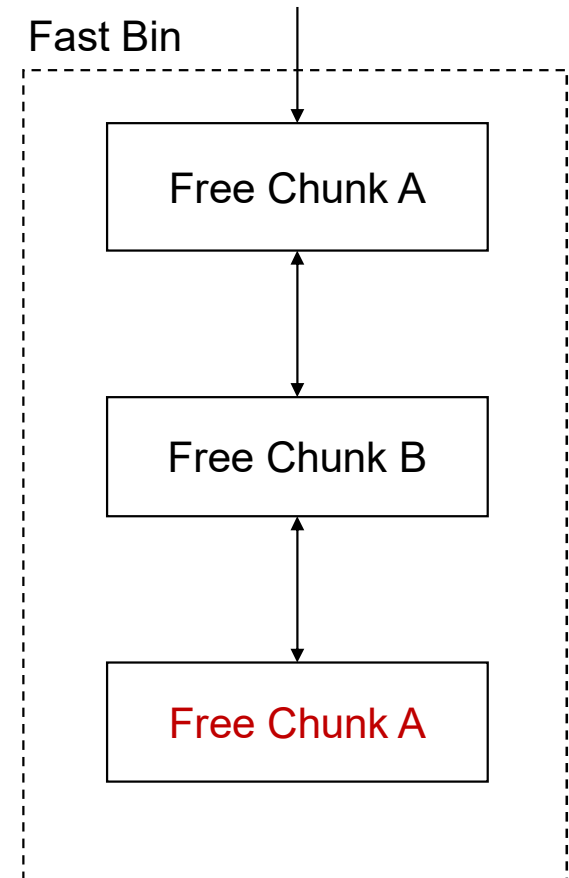
注. 指向已被釋放的內存的指針為空懸指針 (Dangling Pointer)



堆區的其他攻擊：Double-Free

- Double-Free指的是進程**多次釋放統一堆塊**，被多次釋放的堆塊將**被堆管理器分配多次**，最終產生堆塊重疊；

Double-Free多發生在Fast-Bin當中，因為Fast-Bin當中的空閑塊更傾向于被反復分配與釋放



注. Fast-Bin 用于收集較小的空閑堆塊 (16-80B)，方便反復申請小塊內存的場景



堆區的其他攻擊：Heap Over-read

- 堆溢出攻擊越界寫入并覆蓋堆區數據，而Heap Over-Read則直接越界讀出堆區數據，造成信息泄露

著名的Heartbleed Attack是最典型的
Heap Over-Read Attack

- OpenSSL的TLS實現當中，在處理心跳包時未能對長度字段做合理校驗，導致攻擊者可以構造惡意數據包，越界讀取心跳包數據之後的堆區內存；這些內存包含了私鑰等重要信息

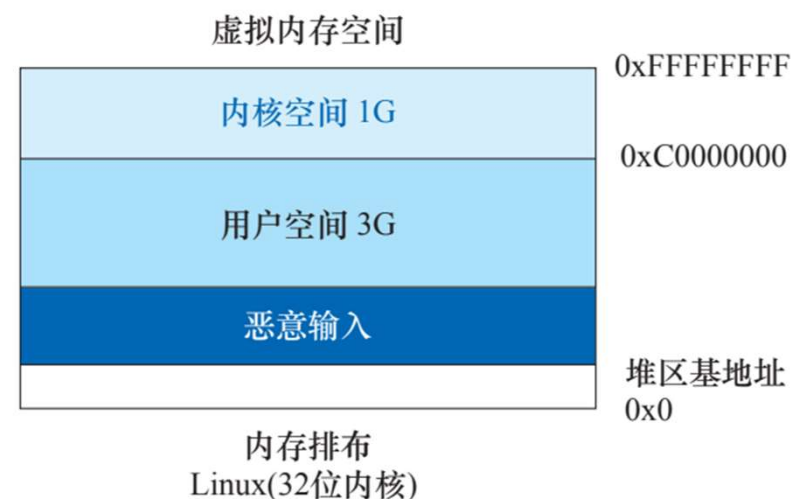


該漏洞可傾瀉的數據量高達**64KB**，而Fast-Bin當中的堆塊最大大小僅為**80B**（為其大小的**800-4000**倍）



堆區的其他攻擊：Heap Spray

- 堆噴（Heap Spray）并非是一種內存攻擊的輔助技術；堆噴申請大量的堆區空間，并將其中填入大量的**滑板指令**（NOP）和攻擊惡意代碼；
- 堆噴使用戶空間存在大量惡意代碼，若EIP指向堆區時將命中**滑板指令區**，受害進程最終將“滑到”惡意代碼



堆噴對抗地址的隨機浮動類型的防禦方案

并實現了惡意代碼的注入



第2節 操作系統基礎防禦方案

- ✓ 2.1 W^X (NX, DEP)
- ✓ 2.2 ASLR
- ✓ 2.3 Stack Canary
- ✓ 2.4 SMAP, SMEP

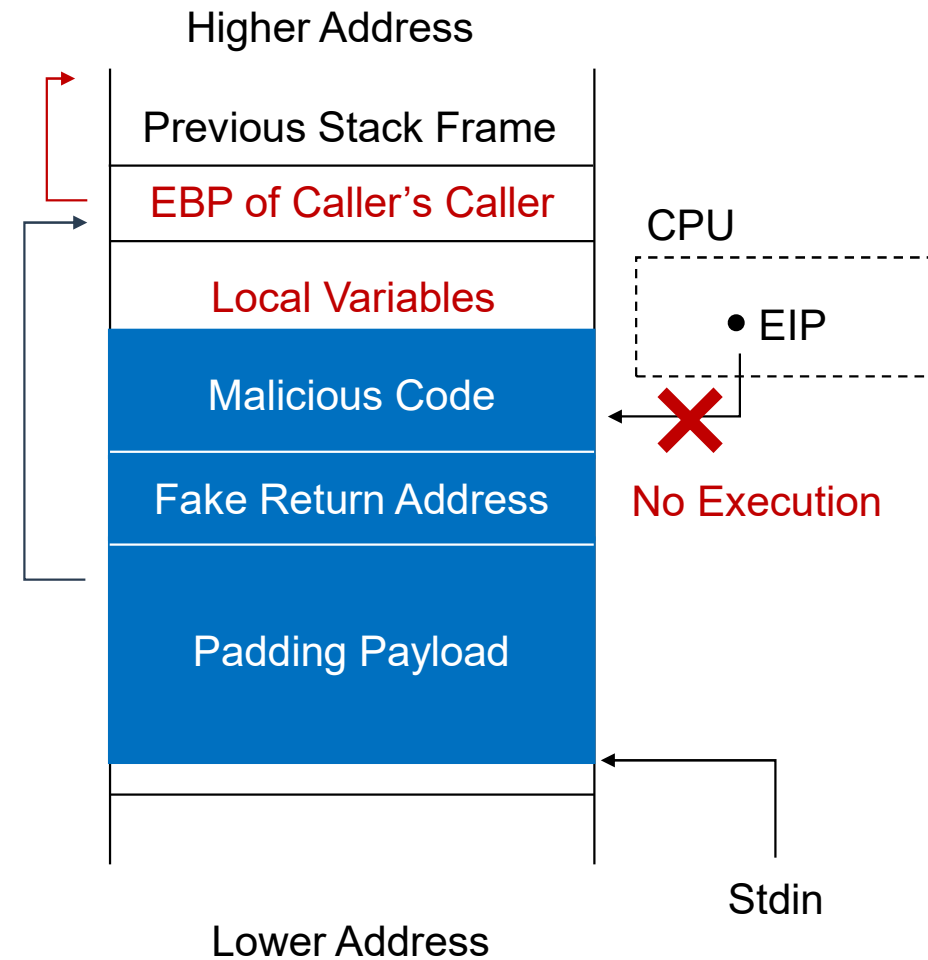


内存防禦技術：W^X

- W^X 是寫與執行不可兼得，即每一個内存頁擁有**寫權限或者執行權限**，不可兼具兩者

當W^X最早在FreeBSD 3.0當中被實現，在Linux下的別名為**NX**（No eXecution），Windows下類似的機制被稱為**DEP**（Data Execution Prevention）

當W^X生效時，**返回至溢出數據的棧溢出攻擊**失效，因為無法執行位于棧區的注入的惡意代碼；
但仍有方法可以繞過NX保護機制，將在下一節的高級控制流劫持方案中介紹



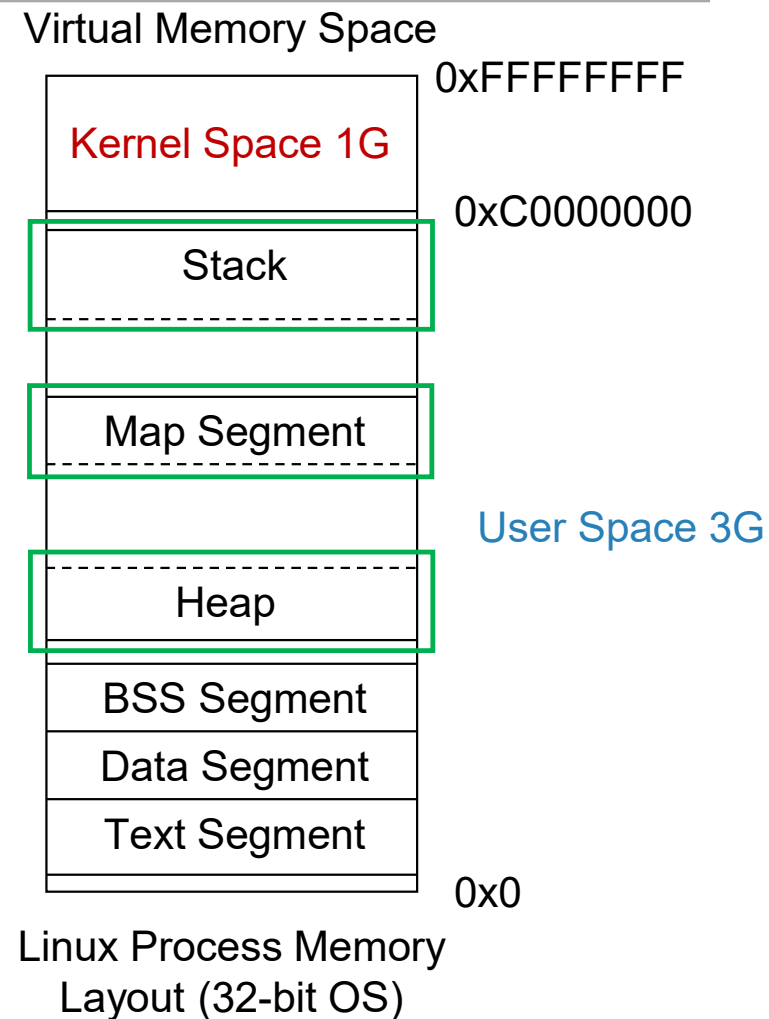


内存防禦技術：ASLR

- ASLR (address space layout randomization) 是一種對虛擬空間當中的**基地址進行隨機初始化**的保護方案；以防止惡意代碼定位進程虛擬空間當中的重要地址；目前在各主流操作系統下均有實現

ASLR隨機化的對象包含了：

- 共享庫的基地址（.so文件加載的基地址）
- 棧區的基地址
- 堆區的基地址



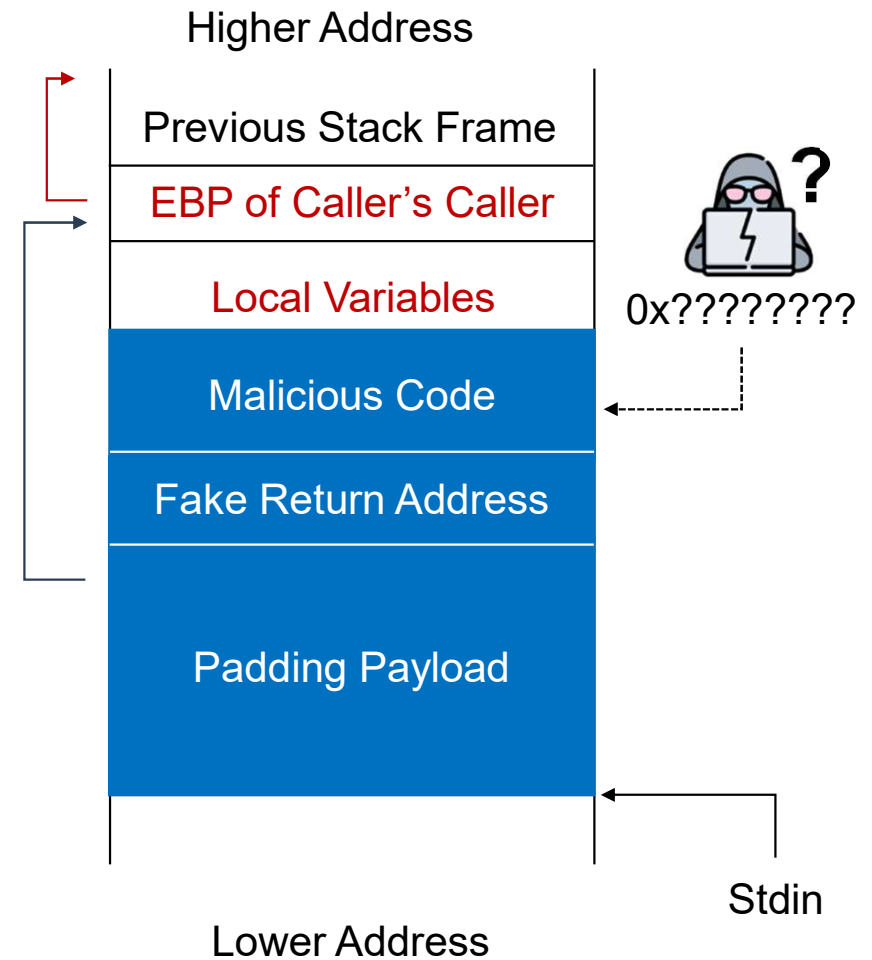
注：ASLR實現需要編譯器的PIE支持（Position Independent Executable）才可將代碼加載到隨機化的地址上



内存防禦技術：ASLR

- ASLR隨機化**棧區基地址**，注入到棧區的惡意代碼内存位置也無法被攻擊者確定

上一節當中**返回至溢出數據的棧溢出攻擊**失效
因為惡意代碼位于棧區，地址被ASLR隨機化，
攻擊者無法確定并寫入惡意代碼的**絕對内存地址**

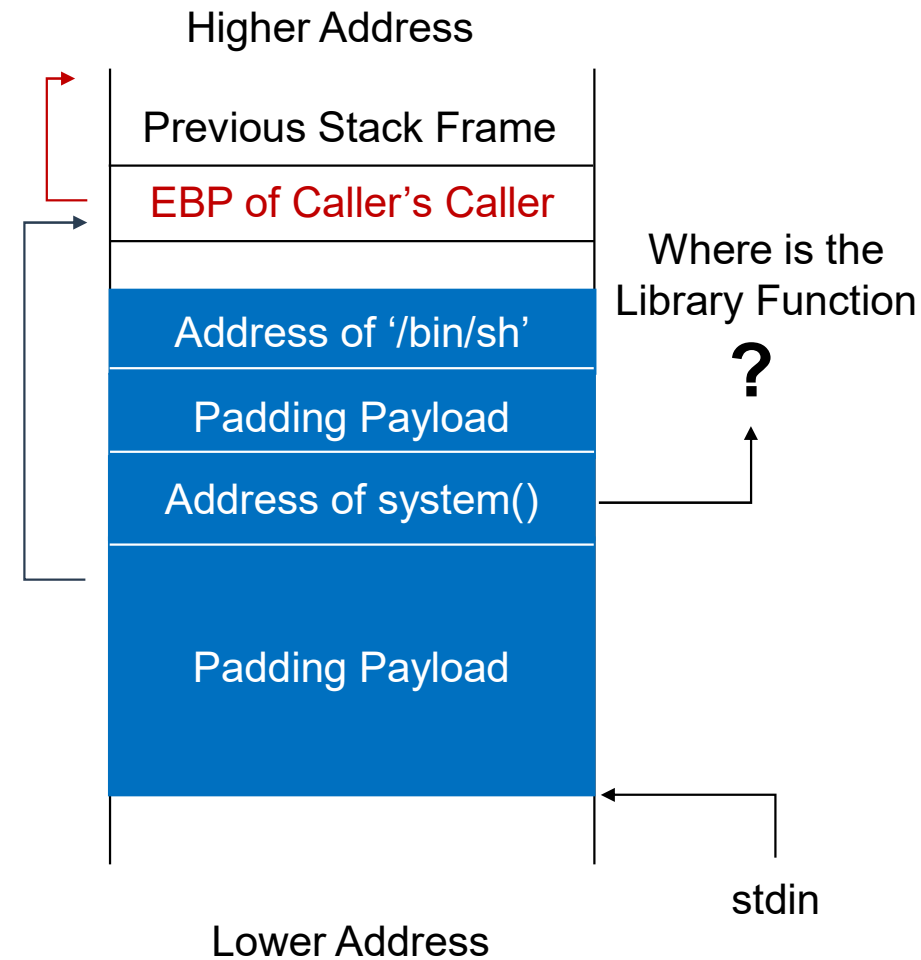




内存防禦技術：ASLR

- ASLR隨機化**共享庫基地址**，庫函數的内存位置也無法被攻擊者確定

上一節當中**返回至庫函數的棧溢出攻擊**失效
因為ASLR將導致攻擊者無法定位目標庫函數





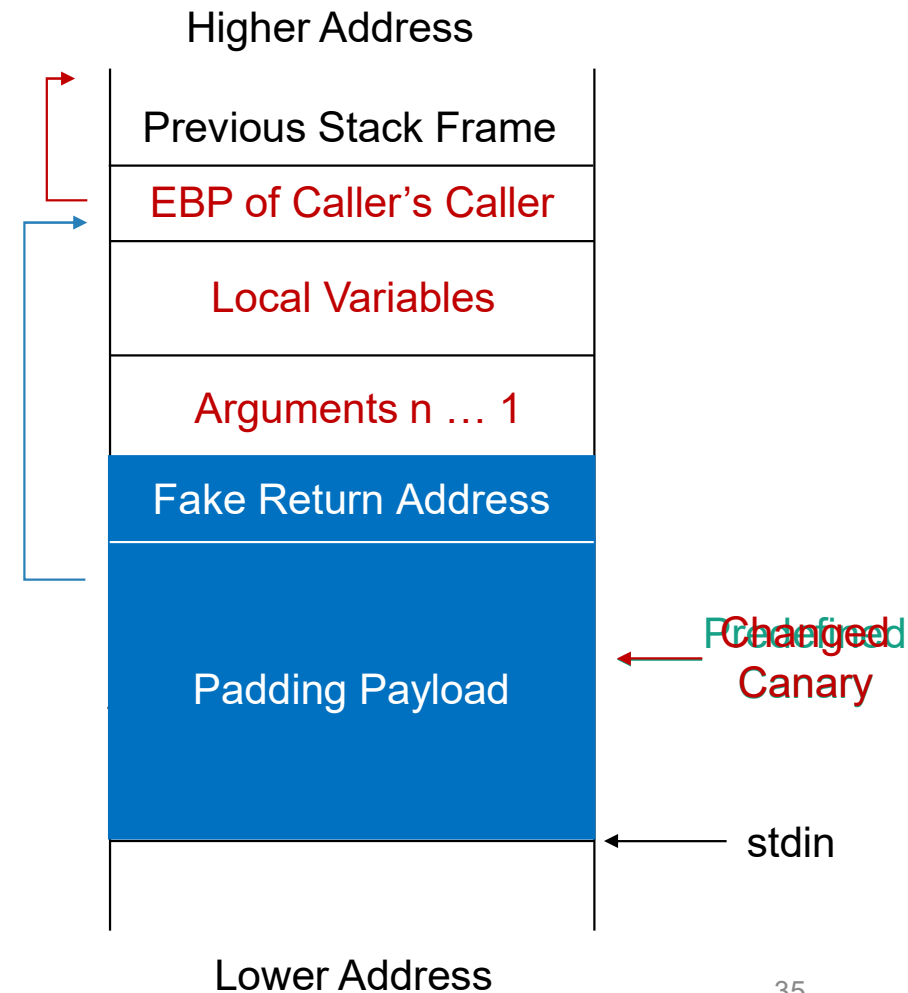
内存防禦技術：Stack Canary

- Canary的本意是金絲雀，Stack Canary是一種防禦棧區溢出的方案

其本質是，在保存的棧幀基地址（EBP）之後插入一段信息，當函數返回時驗證這段信息是否被修改過

當發生棧區溢出時，攻擊者爲了修改返回地址，必須先覆蓋Stack Canary，造成攻擊行爲暴露

注. 其得名緣由：https://en.wikipedia.org/wiki/Stack_buffer_overflow





內存防禦技術：SMAP, SMEP

SMAP和SMEP是兩種基礎的**內存隔離技術**

- SMAP (Supervisor Mode Access Prevention, 管理模式訪問保護) 禁止內核訪問用戶空間的數據
- SMEP (Supervisor Mode Execution Prevention, 管理模式執行保護) 禁止內核執行用戶空間代碼，是預防**權限提升** (Privilege Escalation) 的重要機制

SMAP/SMEP 和 W^X 均需要處理器硬件的支持



内存防禦技術總結

- 雖然操作系統提供了大量防禦內存攻擊的方案，但這些防禦方案仍可以被攻擊者挫敗或繞過：

對於Stack Canary，作為Canary的內容可能被泄露給攻擊者，或被暴力枚舉破解¹

對於ASLR已有去隨機化方案，泄露內存分布信息^{2,3}

在第三節將介紹ROP等進程控制流劫持方案亦可以繞過ASLR、NX的保護機制

1. Wei Wu, et al. "KEPLER: Facilitating Control-flow Hijacking Primitive Evaluation for Linux Kernel Vulnerabilities." 28th USENIX Security Symposium, 2019.

2. Daniel Gruss, et al. "Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR." Proceedings of the 2016 ACM SIGSAC conference on computer and communications security.

3. Ben Gras, et al. "ASLR on the Line: Practical Cache Attacks on the MMU." NDSS. Vol. 17. 2017.



本節相關的前沿研究工作

- 對於堆區管理安全：
GUARDER: A Tunable Secure Allocator¹ 提出一種安全的堆區分配方案，**設計安全的堆管理算法**一直是長時間來難以解決的問題
- 對於棧區管理安全：
Stack Bounds Protection with Low Fat Pointers² **新型棧區內存邊界保護方案**，出發點與Stack Canary類似，均為**保護棧幀邊界防止返回地址篡改**
- 內存保護方案的安全性依賴于微處理器架構安全：
ASLR on the Line: Practical Cache Attacks on the MMU³ 是一種基于**微處理器架構側信道**方案的去除ASLR隨機地址浮動攻擊

1. Sam Silvestro, et al. "Guarder: A tunable secure allocator." 27th USENIX Security Symposium, 2018.

2. Ben Gras, et al. "ASLR on the Line: Practical Cache Attacks on the MMU." NDSS. Vol. 17. 2017.

3. Gregory J., Duck, et al. "Stack Bounds Protection with Low Fat Pointers." NDSS. 2017.



第3節 高級控制流劫持方案

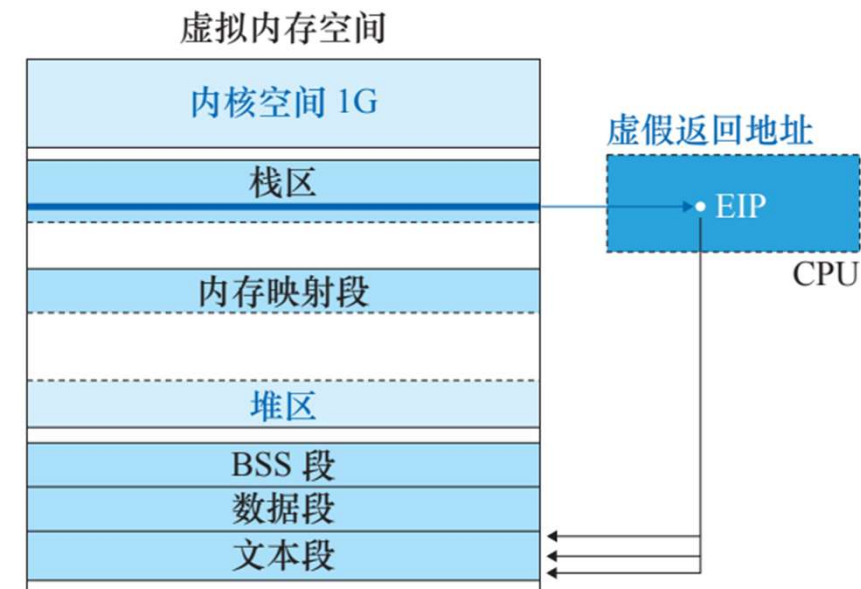
- ✓ 3.1 進程執行的更多細節
- ✓ **3.2 面向返回地址編程**
- ✓ 3.3 全域偏置表劫持
- ✓ 3.4 虛假vtable劫持



面向返回地址编程

- 面向返回地址编程（Return-Oriented Programming, ROP）基于**棧區溢出攻擊**，將返回地址設置為**代碼段中的合法指令**，組合現存指令修改寄存器，劫持進程控制流

ROP利用進程內存空間當中現存的指令，**“編寫”** 惡意程序，劫持進程的控制流





面向返回地址编程

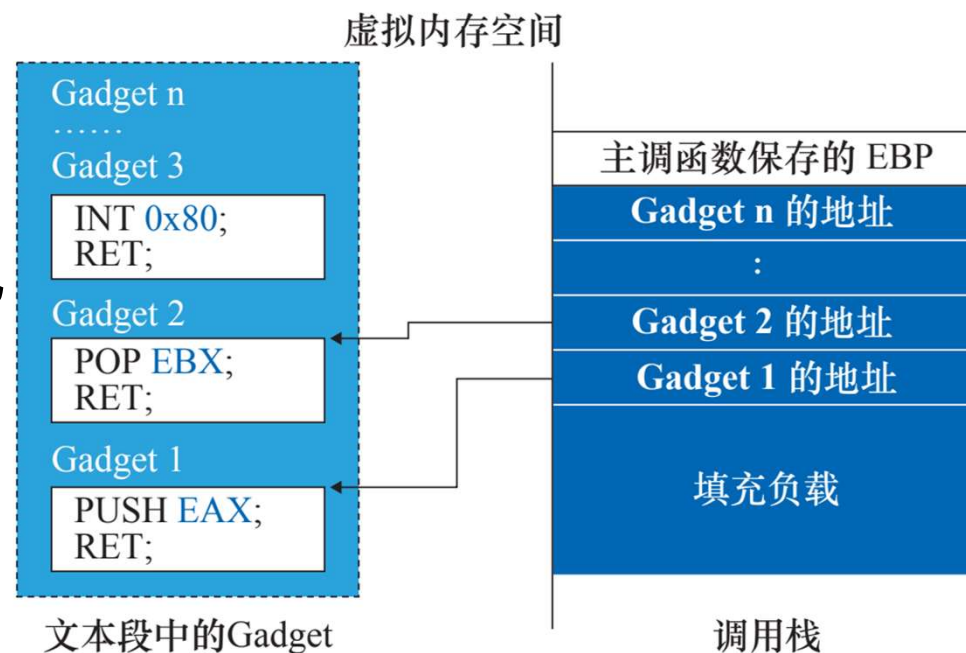
- 面向返回地址编程构造恶意程序所需的指令片段被稱為 **Gadget**

Gadget均以**RET指令**結尾

當一個Gadget執行後，**RET指令**將跳轉執行下一個Gadget

PUSH EAX;
POP EBX;
INT 0x80;
...

← 等价程序



注. Gadget在代碼段，地址固定，可以通過逆向工程工具直接獲得

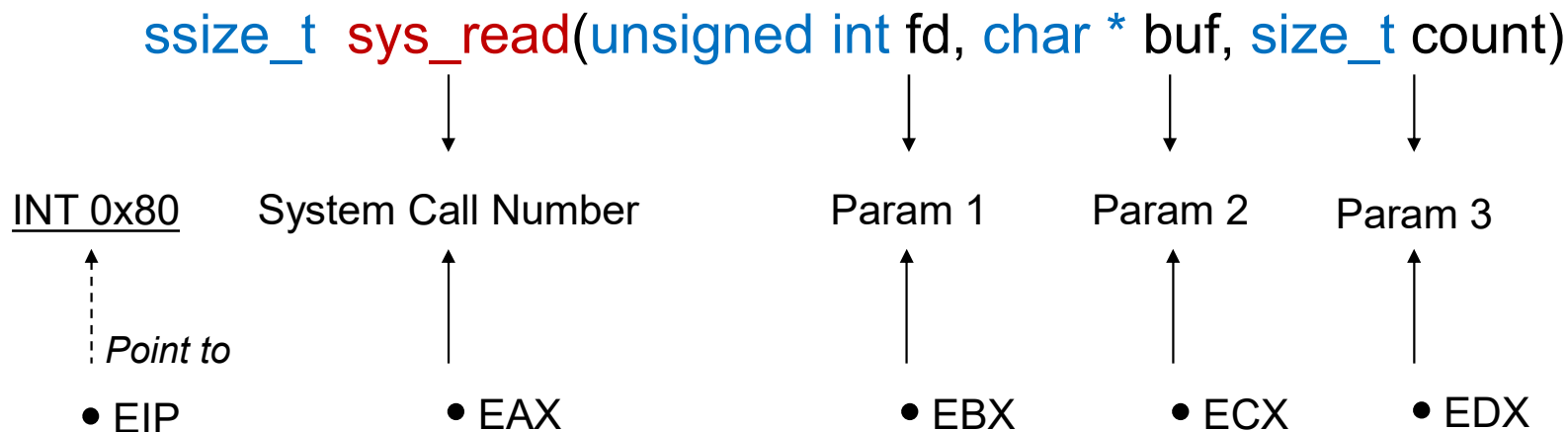


面向返回地址編程

- 示例: 基于ROP的實現的任意系統調用:

核心思想為組合排列Gadget，構造寄存器為系統調用時的狀態如下：

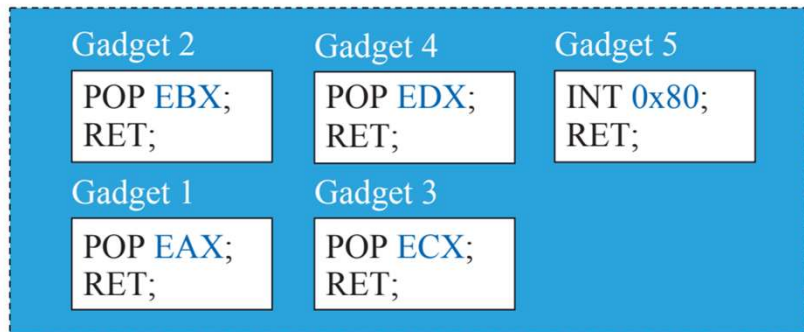
(以sys_read系統調用為例：實現惡意I/O操作)





面向返回地址编程

- 示例: 基于ROP的实现任意系统调用:
- 1. 利用**逆向分析工具**在代码段所搜5个所需的Gadget (如下图所示), 并构造右图中的恶意输入



文本段中的 Gadgets

Gadget 5 的地址
参数3
Gadget 4 的地址
参数2
Gadget 3 的地址
参数1
Gadget 2 的地址
系统调用号
Gadget 1 的地址
填充负载

触发栈溢出的恶意输入



面向返回地址編程

- 示例: 基于ROP的實現的任意系統調用調用:

2. **進行棧區溢出攻擊**，將構造的Payload通過標準輸入流（或網絡流）輸入受害進程，發生棧區溢出，覆蓋返回地址

當被調函數返回時EIP將被賦值為
第一個Gadget的地址

Gadget 5 的地址
参数3
Gadget 4 的地址
参数2
Gadget 3 的地址
参数1
Gadget 2 的地址
系统调用号
Gadget 1 的地址
填充负载

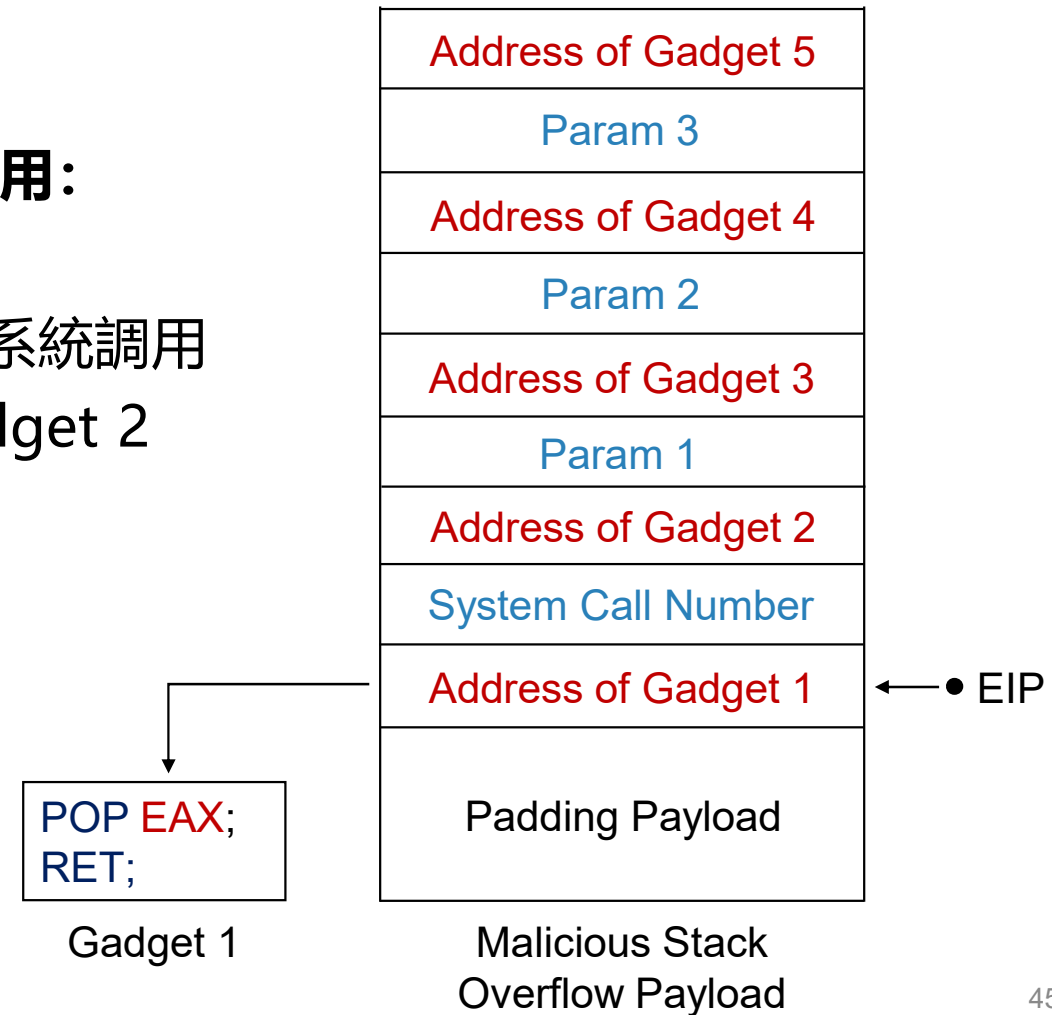
触发栈溢出的恶意输入



面向返回地址編程

- 示例: 基于ROP的實現的任意系統調用調用:

3. EIP執行Gadget 1當中的代碼, **POP**將系統調用號賦值給EAX, **RET**指令執行後EIP指向Gadget 2



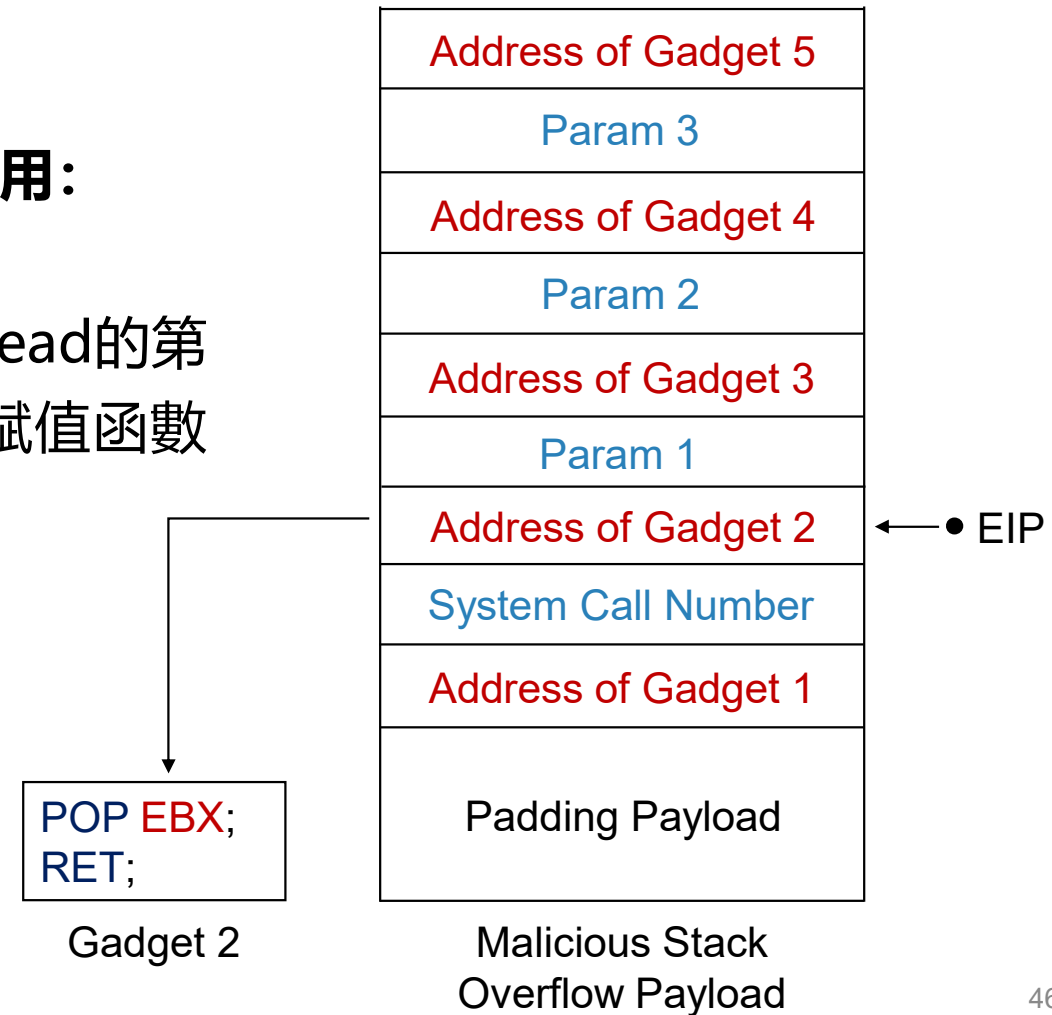


面向返回地址編程

- 示例: 基于ROP的實現的任意系統調用調用:

4. EIP執行Gadget 2當中的代碼，將sys_read的第一個參數：文件描述符賦值至EBX，同理賦值函數的其他參數給寄存器ECX，EDX

完成全部參數賦值後EIP指向Gadget 5

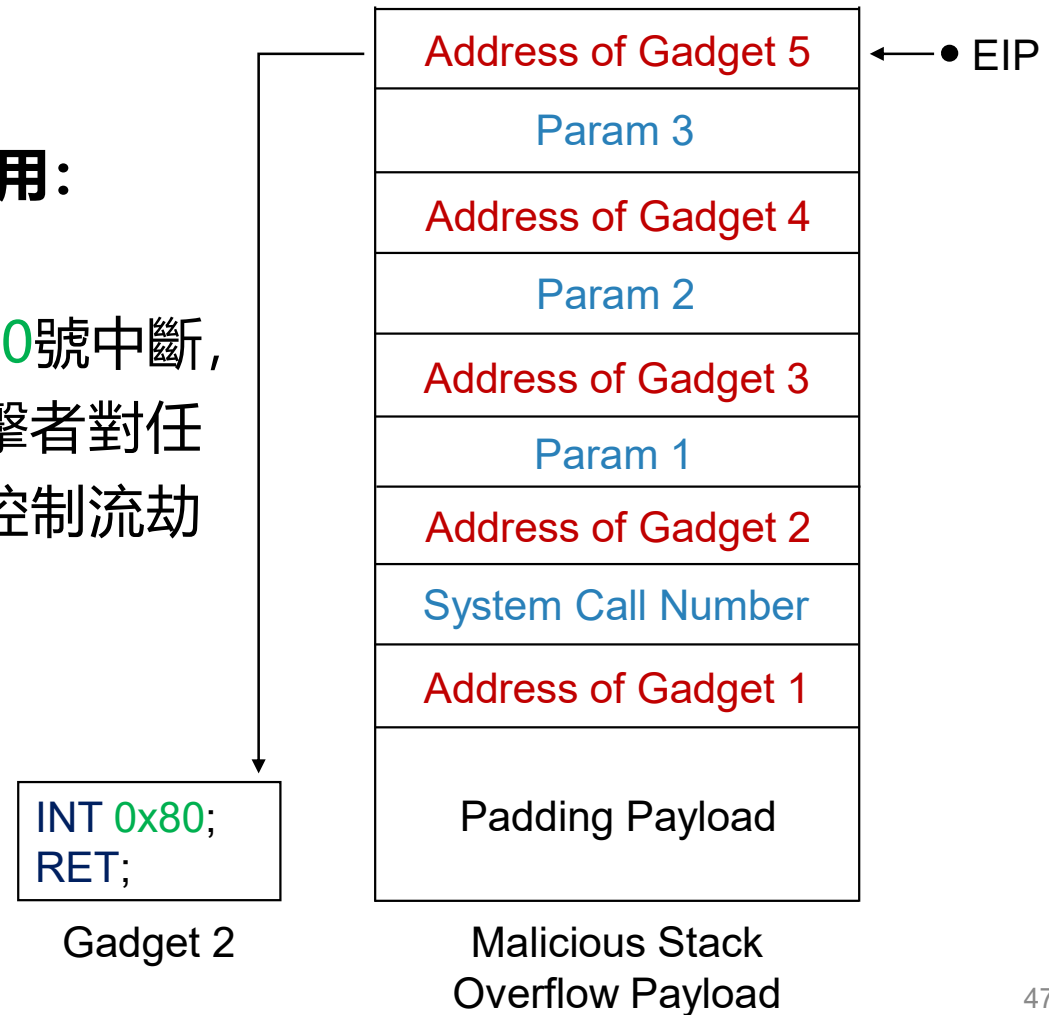




面向返回地址編程

- 示例: 基于ROP的實現的任意系統調用調用:

5. EIP執行 Gadget 5 當中代碼, 觸發0x80號中斷, 將進入內核態進行sys_read系統調用, 攻擊者對任意指定的文件描述符進行讀操作, 完成了控制流劫持

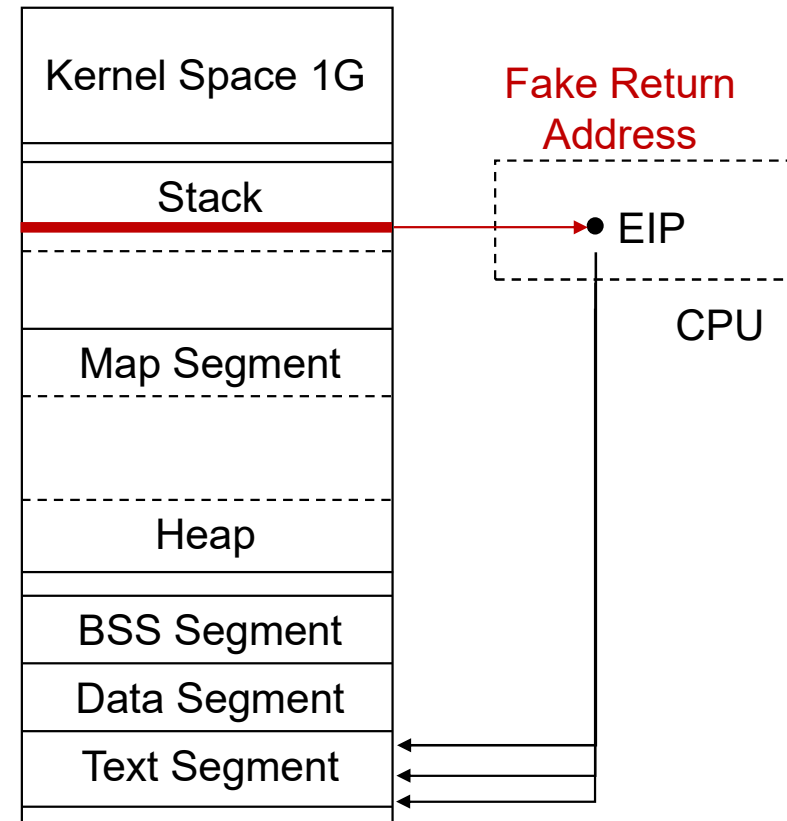




面向返回地址編程的優勢

- 面向返回地址編程的優勢
- 面向返回地址編程（ROP）**可以繞過NX** 防禦機制，因為虛假的返回地址被設置在代碼段（Text Segment），代碼段是存放進程指令的內存區域，必有執行權限
- 面向返回地址編程（ROP）**可以繞過ASLR** 防禦機制，因為惡意代碼並不需直接放在堆區，從而不需要確定地址。

Virtual Memory Space





面向返回地址編程總結

對面向返回地址編程（ROP）攻擊的討論與總結：

- ROP的本質是利用程序**代碼段的合法指令**，重組一個惡意程序，每一個可利用的指令片段被稱作 Gadget，可以說ROP是：a chain-of-gadgets
- ROP可以繞過NX和ASLR防禦機制，但對於Stack Canary則需要額外的信息泄露方案才可繞過這一防禦機制
- ROP使用的Gadget以RET指令結尾；若Gadget的結尾指令為JMP，則為**面向跳轉地址編程**（Jump-Oriented Programming, JOP），原理與ROP類似



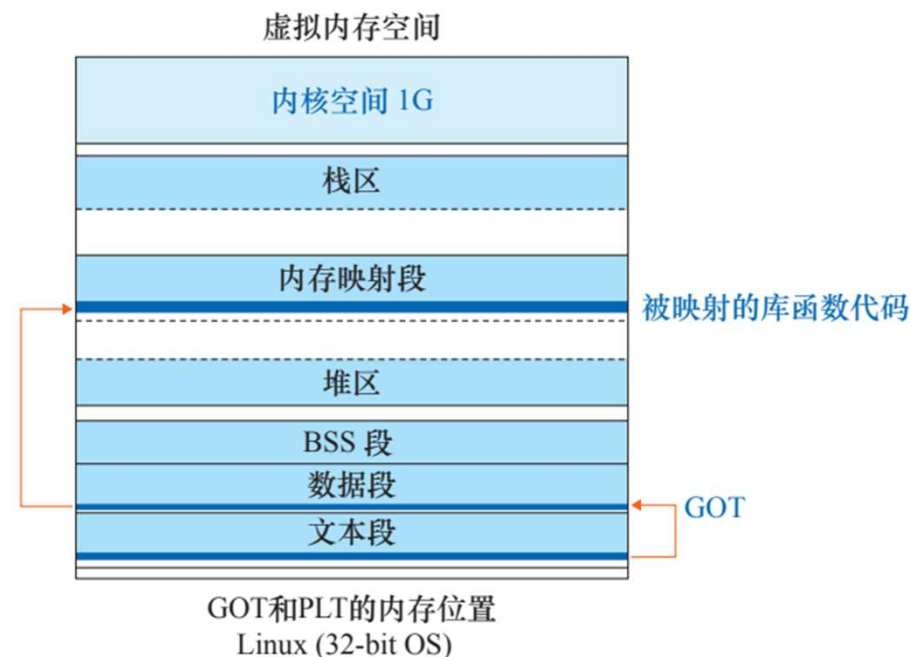
第3節 高級控制流劫持方案

- ✓ 3.1 進程執行的更多細節
- ✓ 3.2 面向返回地址編程
- ✓ **3.3 全域偏置表劫持**
- ✓ 3.4 虛假vtable劫持



全域偏移表與程序鏈接表

- 為了使進程可以找到內存中的動態鏈接庫，需要維護位于**數據段**的**全域偏移表**（Global Offset Table, GOT）和位于**代碼段**的**程序鏈接表**（Procedure Linkage Table, PLT）
- 程序使用**CALL指令**調用共享庫函數；其調用地址為**PLT表地址**，而後由PLT表**跳轉索引GOT表**，GOT表項指向內存映射段，也就是位于動態鏈接庫的庫函數



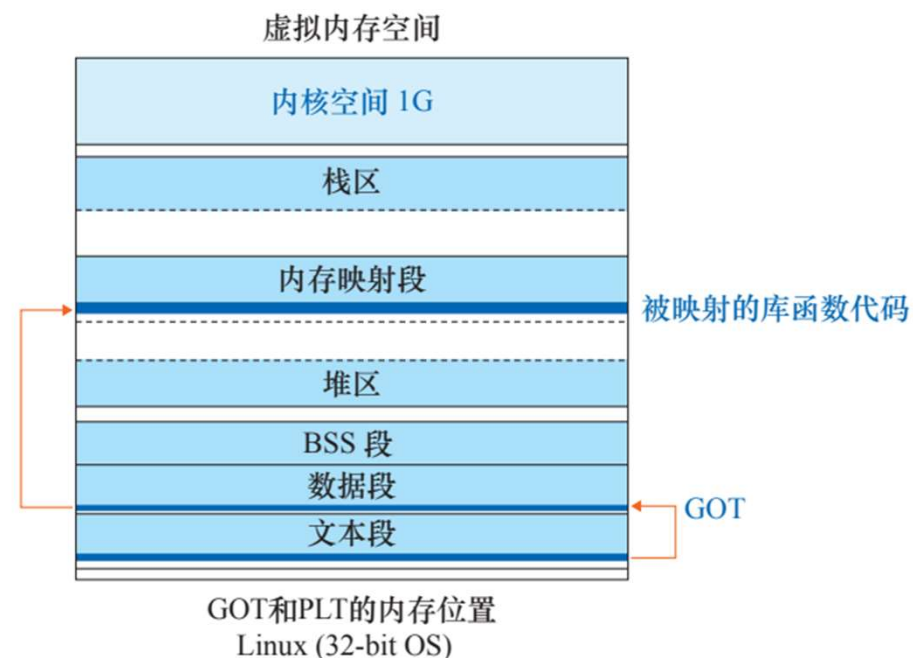


全域偏移表與程序鏈接表

- PLT表在運行前確定，且在程序運行過程中不可修改（Text Segment 不可寫）

GOT表根據一套“惰性的”共享庫函數加載機制，GOT表項在庫函數的首次調用時確定，指向正確的內存映射段位置

- **動態鏈接器**將完成共享庫在的映射，並為GOT確定表項

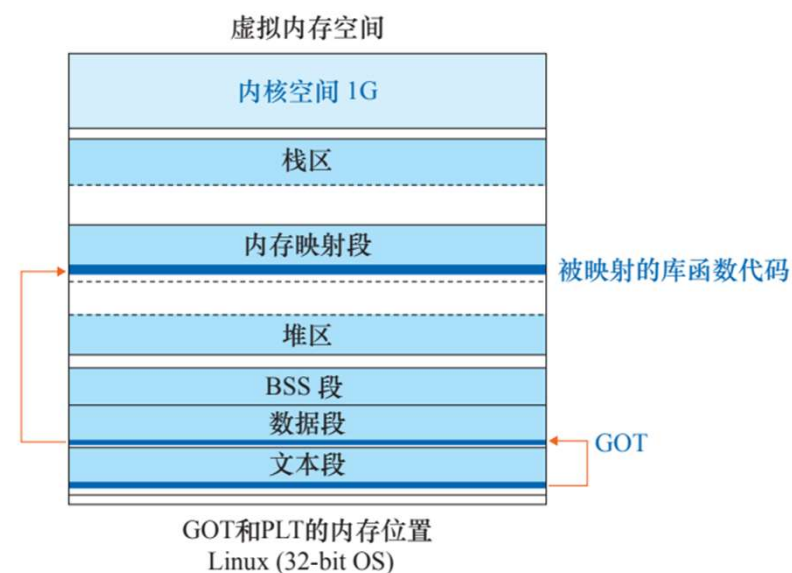


注. 動態鏈接器為最早被加載的動態鏈接庫



全域偏移表與程序鏈接表

- PLT表不直接映射共享庫代碼位置的原因有二：
 1. ASLR將隨機浮動共享庫的基地址，導致共享庫的位置無法被硬編碼
 2. 并非動態鏈接庫當中的所有庫函數都需要被映射（降低內存開銷）

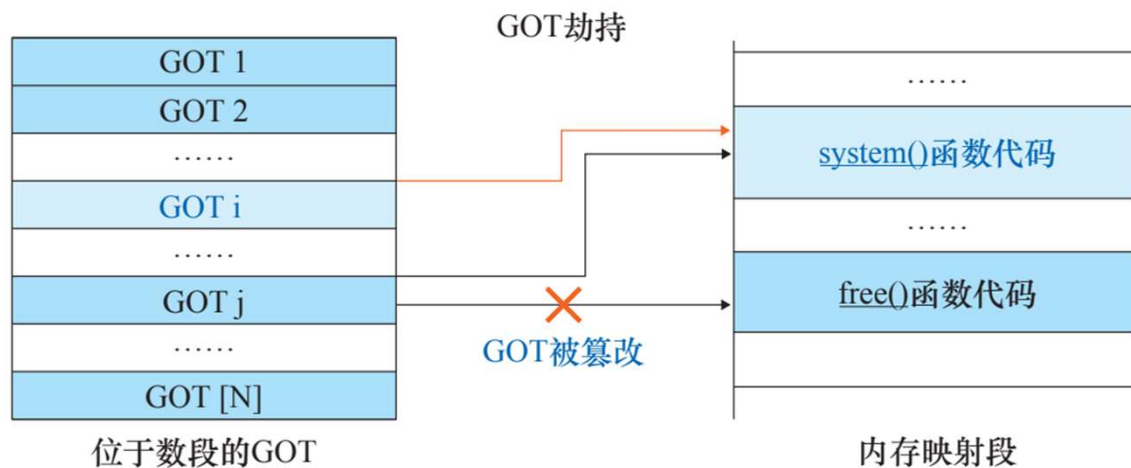




全域偏移表劫持

- GOT Hijacking (全域偏置表劫持)攻擊的本質是惡意篡改GOT表，使進程調用攻擊者指定的庫函數，實現控制流劫持

如圖，當GOT表被修改後，當攻擊者調用free函數時，實際上將調用system函數





全域偏移表劫持的優勢

- **全域偏置表劫持的優勢：**
- **GOT Hijacking攻擊可以繞過NX保護機制。** 因為裝載共享庫函數的頁上必須有可執行權限；且GOT表位于數據段，數據段可讀可寫
- **GOT Hijacking攻擊可以繞過ASLR保護機制。** 因為ASLR無法隨機化代碼段的位置，攻擊者仍然可以通過PLT表惡意讀取GOT表項目，而後得到動態庫當中函數的地址



全域偏移表劫持步驟

- GOT Hijacking可以分爲大致如下的幾個步驟：
 1. 攻擊者通過讀PLT確定要被修改的受害GOT表項的地址
 2. 攻擊者讀取這一GOT表項，得到任一庫函數的內存映射段地址
 3. 攻擊者將得到的地址加一個偏置，得到希望被惡意調用的函數的內存映射段地址
 4. 攻擊者將這一地址寫入定位好的受害GOT表項當中

其中，惡意讀寫GOT表項既可以通過基于棧溢出的ROP來實現，也可以通過基于堆溢出的任意位置讀寫來實現



全域偏移表劫持總結

- **GOT Hijacking的總結:**
- GOT Hijacking本質上是一種修改GOT表項來實現的控制流劫持；這種攻擊方案要依賴于棧區溢出等基礎攻擊方式才可以實現
- 這種攻擊方案可以繞過NX與ASLR防禦機制
- 目前已有RELRO (read only relocation) 機制，可將GOT表項映射到只讀區域上，一定程度上預防了對GOT表的攻擊



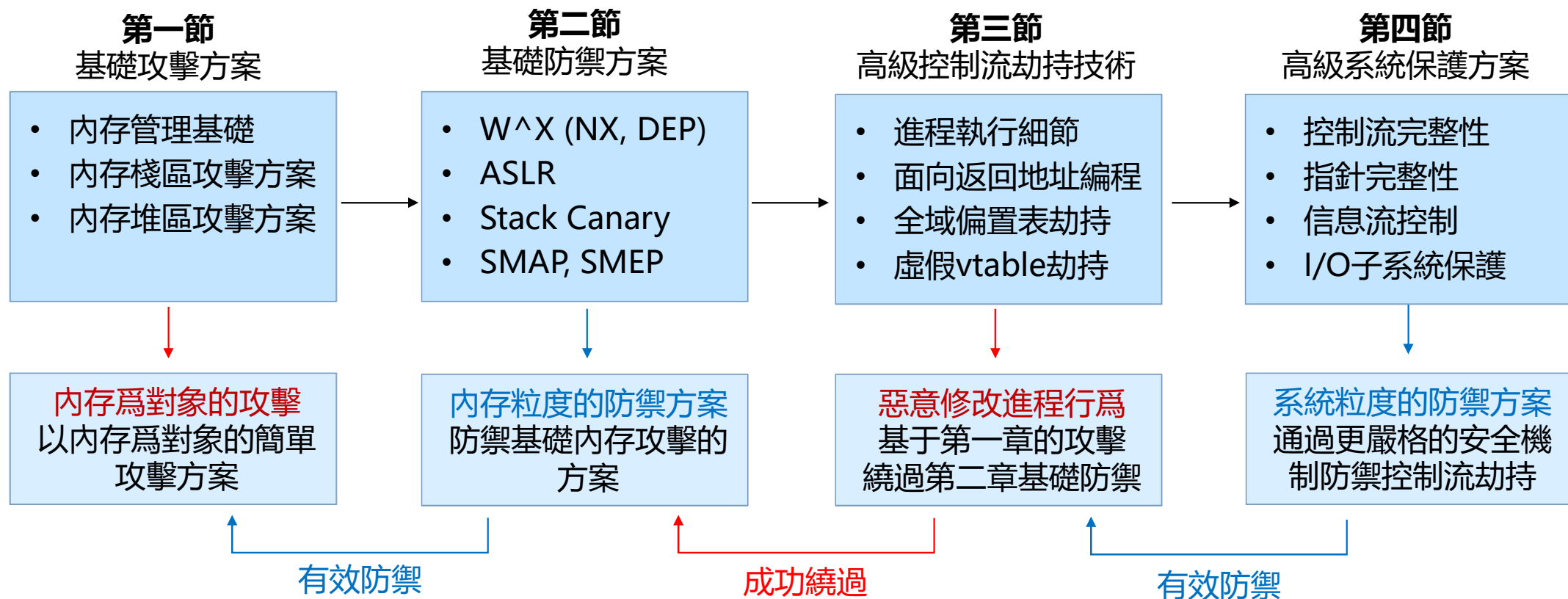
第5節 總結和展望





總結

本章當中，我們首先回顧了最早的操作系統安全問題，Morris Worm；并分析了 SQL Slammers 侵染SQL Server的全過程，之後由淺入深地介紹了操作系統下一系列**攻擊**與**防禦**方案





展望

宏觀來看操作系統安全研究發展趨勢有二：

- 其一，對於操作系統的防禦與攻擊方案**與新型硬件特性**加以結合，例如：

Gras, et al.¹ 在2017年提出了一種利用硬件緩存側信道去除ASLR隨機化的方法

Frassetto, et al.² 通過拓展指令集，實現了內存隔離，防止了非法內存訪問的發生

借助SGX，MPK等新型硬件安全功能，借助硬件虛擬化基礎設施實現新的系統安全保障機制研究目前是一個**活躍的研究分支**

1. Ben Gras, et al. "ASLR on the Line: Practical Cache Attacks on the MMU." NDSS. Vol. 17. 2017.

2. Tommaso Frassetto, et al. "IMIX: In-Process Memory Isolation EXtension." 27th USENIX Security Symposium, 2018.



展望

宏觀來看操作系統安全研究發展趨勢有二：

- 其二，操作系統環境下的**漏洞的自動化挖掘**

Wang, et al.¹ 實現了一種對於操作系統缺乏二次檢查漏洞的全自動化挖掘方法，其使用的傳統數據流與控制流分析，類似的工作還有很多

Jia, et al.² 針對於內核驅動當中的未初始化漏洞進行了分析

Eckert, et al.³ 利用模型驗證（Model Checking）方法實現了對於堆管理器的安全性驗證

1. Wenwen Wang, et al. "Check it again: Detecting lacking-recheck bugs in os kernels." in Proceedings of CCS, 2018.

2. Xiangkun Jia, et al. "Towards efficient heap overflow discovery." 26th USENIX Security Symposium, 2017.

3. Moritz Eckert, et al. "Heaphopper: Bringing bounded model checking to heap implementation security." 27th USENIX Security Symposium, 2018.



展望

宏觀來看操作系統安全研究發展趨勢有二：

- 其二，操作系統**漏洞的自動化挖掘**

目前系統相關漏洞自動化挖掘是極其活躍的研究分支

然而，其中具有顯著成效的方法均以傳統的**靜態源代碼分析**為主，具有更好漏洞挖掘效果的**動態分析方案**（例如模糊測試，Fuzzing）在應用到操作系統這類高度複雜軟件系統時仍然存在各種各樣的局限性

目前來看，精准全面的操縱系統漏洞的自動化挖掘
仍然是一個 “**美好的理想**”